Using Graph Neural Network to Solve the Traveling Salesman Problem

Pingbang Hu, Jonathan Moore, Yi Zhou, Shubham Kumar Pandey, Anuraag Ramesh

University of Michigan

Abstract

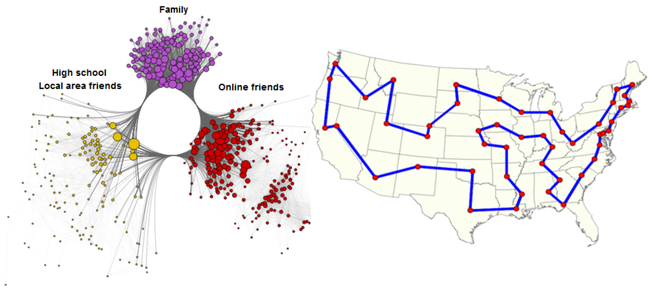
Graph neural network is a rising concept in machine learning that has demonstrated significant potential in many areas. Our project aims to further explore this potential by applying it to the classic **Traveling Salesman Problem (TSP)**. We explore a novel concept called **imitation learning**, which is a special type of **reinforcement learning**.

The former works focuses on fixed size training with large size instances. In this work, we focus on generalization ability of model trained on small instances instead. Specifically, we train on TSP with 15 nodes, which translate into an **Integer Linear Programming (ILP)** with hundreds of variables and several hundreds of constraints. Then, we test on TSP instances with instances with various sizes and see the generalization ability of this novel approach.

The Basics of GNN

A graph neural network (GNN) is a class of neural network for processing data best represented by graph structures. They were popularized by their use in supervised learning on properties of various molecules from the natural of this task: 3d graph structure.

Since their inception, variants of the message passing neural network (MPNN) framework have been proposed. These models optimize GNNs for use on larger graphs and apply them to domains such as social networks, citation networks, and online communities. GNNs have also been relatively successful in various NP-hard combinatorial problems, automated planning and path-planning areas due to the inherent graph structure of data.



Problem Formulation

One way to formulate TSP is by **Integer Linear Programming**. Given an undirected weighted graph $\mathcal{G} = (\mathcal{E}, \mathcal{V})$ with nodes being $1, \ldots, n$, and define

$$x_{ij} \coloneqq \begin{cases} 1, & \text{if } (i,j) \in \mathcal{E}^* \\ 0, & \text{if } (i,j) \in \mathcal{E} \setminus \mathcal{E}^*, \end{cases}$$

where $\mathcal{E}^* \subset \mathcal{E}$ is a valid path in this TSP instance. This means that x_{ij} acts like an **indicator** variable. We further denote the weight on edge (i,j) by c_{ij}, then for a TSP problem instance, we can formulate the problem by so-called **Miller–Tucker–Zemlin formulation**.

A popular way to solve an ILP is to first **relax** the integrality constraints by letting $x_{ij} \in [0,1]$. Then, we can divide the original relaxed LP into two sub-problems by **splitting the feasible region** according to **a chosen variable** that violates integrality constraints in the current relaxed LP solution $\boldsymbol{x}^*$, say $\boldsymbol{x}_i^*$, then

$$\boldsymbol{x}_i \le \lfloor \boldsymbol{x}_i^* \rfloor \vee \boldsymbol{x}_i \ge \lceil \boldsymbol{x}_i^* \rceil, \qquad \exists i \le p \mid \boldsymbol{x}_i^* \notin \mathbb{Z}.$$

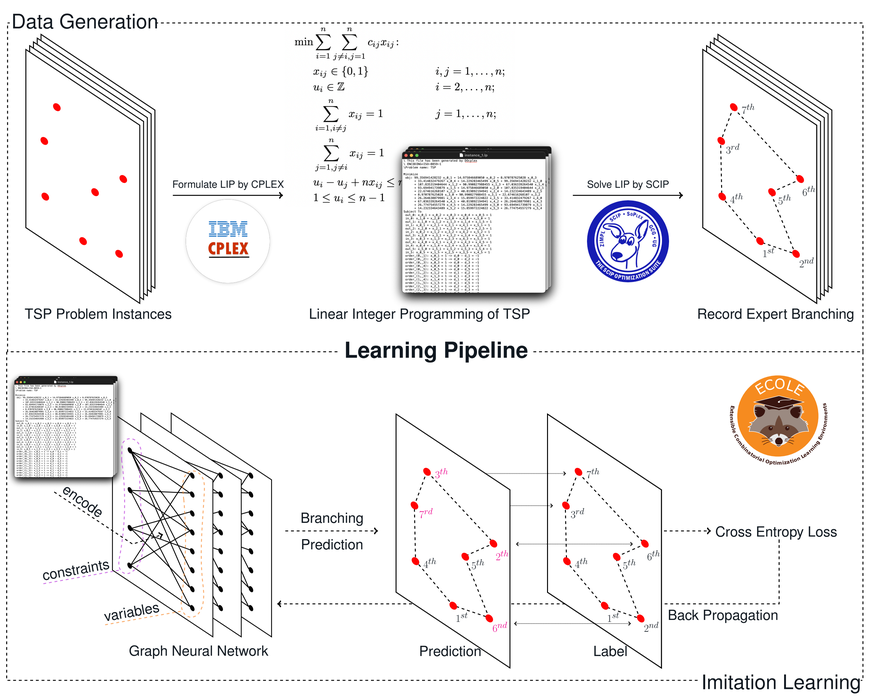
are two additional constraints in two sub-problems respectively. This gives us a recursive algorithm called **Branch and Bound**.

Learning Algorithm

A good branching decision can reduce the sub-problem size, hence the solving time significantly since we can estimate the lower bound after each branching of the optimal cost. Then, after a branching, we can eliminate those branching choices which are not going to produce a better tour.

Our goal now is to learn how to branch. This is a sequential decision problem, hence, we naturally model this as a **Markov Decision Process (MDP)** problem. Suprisingly, this branching decision problem naturally has an **graph structure**, hence, rather than model TSP by a graph from nodes' coordinates, we **model the problem as a graph from a way deeper viewpoint**.

Our learning pipeline is as follows. We first create some random TSP instances, and turn it into ILP. Then, we use imitation learning to learn how to choose the **branching target** at each branching. Our GNN model will produce a set of action with the probability corresponding to each possible action, in this case, which variable to branch. We then use **Cross Entropy Loss** to compare our prediction to the result produced by `SCIP` and complete one iteration.



Specifically, we will pass TSP instances to `SCIP`, which is a modern SOTA MILP (Mixed Integer Linear Programming) solver, to get all the solving states in order to solve MDP problem. The modern solver usually use mixed branching strategy to balance the running time, and a well-known costly but strong strategy is called **strong branching**, and in order to learn the strongest branching strategy, we make `SCIP` to use strong branch with probability a half. This learning process is called **imitation learning**, or **behavioral cloning**.

Mathematically, we first run the SOTA solver `SCIP` to get state-action pairs $\mathcal{D} = \left\{(s_i, \boldsymbol{a}_i^*)\right\}_{i=1}^N$, where the states is the solver's branching strategy, and the action set contains what variables we can branch on.

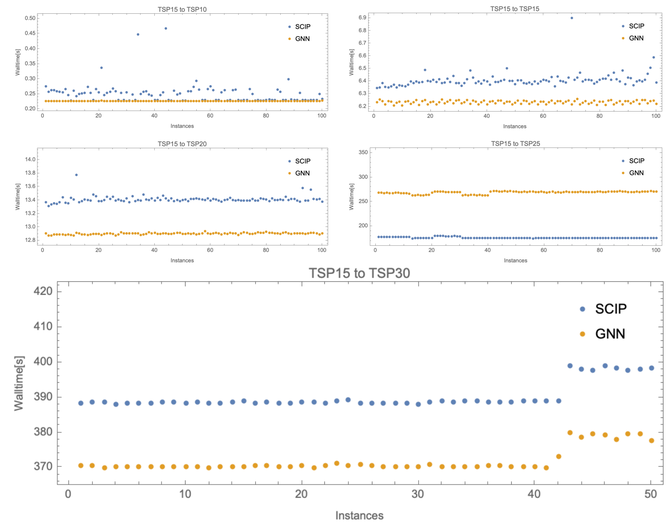
To learn the policy $\widetilde{\pi}^*$, we minimize the cross-entropy loss between our branching prediction and the solver's branching choice:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{(\boldsymbol{s}, \boldsymbol{a}^*) \in \mathcal{D}} \log \widetilde{\pi}_\theta(\boldsymbol{a}^* \mid \boldsymbol{s}).$$

After learning, we evaluate our model on TSP instances with various sizes to see the generalization ability. Specifically, we use Ecole to do the evaluation. We configure `SCIP` to its default strategy and compare the result to our learned branching strategy by looking at the time needed to solve this particular TSP instance.

Experimental Result

We test the generalization ability of our model trained with TSP15 on various sizes TSP instances on GreatLakes with one A100 GPU and 8GB, 16 cores CUP. The figures below plots the walltime needed for our model and `SCIP` to solve a particular TSP instance on y-axis, and we do this for 100 instances for every testing size.



We see that our model outperform the baseline on TSP15, and **successfully generalize to TSP10, TSP20 and TSP30**.

However, there is a counter-intuitive result, namely on TSP20 and TSP30. We see that our model does not outperform the baseline, and is behind by 50% in average; but our model does outperform the baseline by 4.70%. A potential explanation is that our learned model is more stable and robust than the baseline's heuristic, hence we can still conclude that our model **do** generalize great on larger or smaller instances.

Conclusion

Unlike other works which focuses on just to solve TSP approximately, we focus on generalization ability of solving TSP **exactly**.

With limited computation power and training on small size instances, our result shows a promising performance on generalization ability of branch, which is a hard combinatorial optimization problem and typically hard to generalize.

The significance of our works is pretty straightforward: We're now confidence to generalize on larger instances by combining various machine learning techniques like standard reinforcement learning, deep learning, transformer, etc.

Future Work

Due to computational limitation, we can't train on larger instances. One may further generalize our work by training on larger instances, for instance, TSP50, TSP100 or higher.

On the other hand, our work should be easy to reproduce and be combined with various of standard reinforcement technique to further fine-tuned our trained model.